

trompet

A [Typst](#) package for making Tromp lambda diagrams, built on top of [Lambdabus](#) and [CeTZ](#)

github.com/CrowdingFaun624/trompet

Version 0.1.1

Guide

trompet	1
Tromp Diagram	2
Styling	2
Expression Functions	3

Reference

Functions	5
abstraction	5
application	5
expression	6
tromp	6
value	7

Tromp Diagram

There are three ways to call `tromp` to display a Tromp diagram.

```
#tromp("\\f.\\n.f (f n)")      #import "@preview/      #tromp(expression(
                                lambdabus:0.1.0" as lmd          abstraction("f",
                                                              abstraction("n",
                                                              application(
                                                              "f", "f n"
                                ))
                                )
                                )
                                ))
```

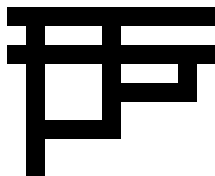
All of the above produce the following Tromp diagram (the Church numeral 2):



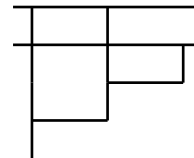
Styling

Tromp diagrams can use the `"pixel"` mode or the `"line"` mode.

`"pixel"`



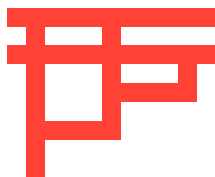
`"line"`



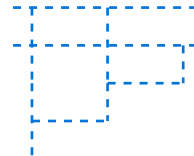
```
#tromp("\\f.\\n.f (f n)", mode: "pixel")
```

```
#tromp("\\f.\\n.f (f n)", mode: "line")
```

Tromp diagrams can adjust the styling of the fill of the pixels or the stroke of the lines using any [CeTZ style](#).

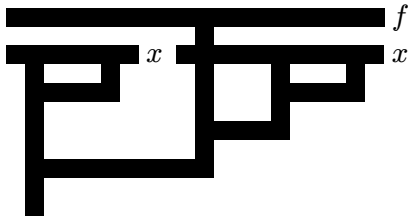


```
#tromp("\\f.\\n.f (f n)", style: red)
```

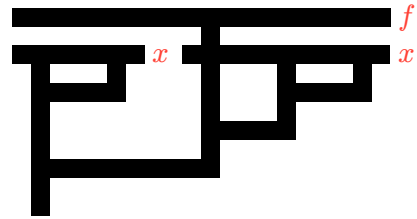


```
#tromp(
  "\\f.\\n.f (f n)",
  mode: "line",
  style: (paint: blue, dash: "dashed")
)
```

Tromp diagrams can label abstractions.



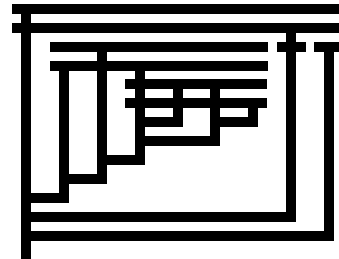
```
#tromp("λf.(λx.x x) (λx.f (x x))",  
labels: auto)
```



```
#tromp(  
  "λf.(λx.x x) (λx.f (x x))",  
  labels: param => {  
    text($italic(param)$, red)  
  }  
)
```

The scale of a Tromp diagram can be adjusted using the scale parameter.

```
#tromp(  
  "λn.λf.n (λf.λn.n (f (λf.λx.n f (f  
x)))) (λx.f) (λx.x)",  
  scale: 0.5,  
)
```



Expression Functions

Lambda expressions can be expressed using the `expression()`, `value()`, `abstraction()`, and `application()` functions instead of strings.

`value()`, `abstraction()`, and `application()` all take a style parameter which behaves the same way as above.

The `expression()` function returns an expression with additional metadata which allows Lambdabus to process it.

```
>>> #abstraction("a", value("a"))  
(  
  type: "abstraction",  
  param: "a",  
  body: (type: "value", name: "a", style:  
auto),  
  style: auto,  
  label: auto,  
)  
>>> #expression(  
  abstraction("a", value("a"))  
)  
(  
  type: "abstraction",  
  param: "a",  
  body: (type: "value", name: "a", style:  
auto, index: 0),  
  style: auto,  
  label: auto,  
  index: 0,  
)
```

The `value()` function takes in a single string for the parameter name, referring to a corresponding abstraction with the same parameter.

The `abstraction()` function takes in a string for the parameter name and a lambda expression representing the body of the abstraction. Additionally, it takes a `label` parameter which can be `auto`, `none`, a function from the parameter name to content, or just content.

The `application()` function takes in a lambda expression for the function and a lambda expression for the parameter of the application.

Functions

- [abstraction\(\)](#)
- [application\(\)](#)
- [expression\(\)](#)
- [tromp\(\)](#)
- [value\(\)](#)

abstraction

Creates a stylable abstraction.

Parameters

```
abstraction(  
    parameter: str,  
    body: str dictionary,  
    style: auto color stroke dictionary,  
    label: auto none content function  
)
```

parameter `str`

The string corresponding to the parameter of this abstraction.

body `str` or `dictionary`

An expression constructed through [value\(\)](#), [abstraction\(\)](#), and [application\(\)](#) or a Lambdabus expression.

style `auto` or `color` or `stroke` or `dictionary`

If using the "pixel" mode, must be `auto` or a color. If using the "line" mode, must be `auto`, a stroke, or a [CeTZ style](#). If auto, defaults to the whole diagram's default style.

Default: `auto`

label `auto` or `none` or `content` or `function`

`auto`, `none`, `content`, or a function that takes the parameter as a string and returns content. If `auto`, defaults to the whole diagram's label style.

Default: `auto`

application

Creates a stylable application.

Parameters

```
application(  
  function: str dictionary ,  
  parameter: str dictionary ,  
  style: auto color stroke dictionary  
)
```

function str or dictionary

An expression constructed through `value()`, `abstraction()`, and `application()` or a Lambdabus expression.

parameter str or dictionary

An expression constructed through `value()`, `abstraction()`, and `application()` or a Lambdabus expression.

style auto or color or stroke or dictionary

If using the "pixel" mode, must be `auto` or a color. If using the "line" mode, must be `auto`, a stroke, or a [CeTZ style](#). If auto, defaults to the whole diagram's default style.

Default: `auto`

expression

Adds additional metadata to an expression made with `value()`, `abstraction()`, or `application()` so that Lambdabus can do math on it.

Parameters

```
expression(expression: dictionary)
```

expression dictionary

The lambda calculus expression constructed through `value()`, `abstraction()`, or `application()`.

tromp

Creates a Tromp diagram.

Parameters

```
tromp(  
  expression: str dictionary ,  
  mode: str ,  
  scale: float ,  
  style: color stroke dictionary ,  
  labels: content auto none  
) -> content
```

expression str or dictionary

A [Lambdabus expression](#) or result of `expression()`.

mode str

Either the string "pixel" or "line".

Default: "pixel"

scale float

A float representing the scale from the default pixel size.

Default: 1.0

style color or stroke or dictionary

If using the "pixel" mode, must be a color. If using the "line" mode, must be a stroke or a [CeTZ style](#).

Default: black

labels content or auto or none

auto or a function that takes in a parameter name string and returns content.

Defaults to param => `$italic(param)$`.

Default: none

value

Creates a stylable value.

Parameters

```
value(  
  parameter: str ,  
  style: auto color stroke dictionary  
)
```

parameter `str`

The string corresponding to the abstraction parameter.

style `auto` or `color` or `stroke` or `dictionary`

If using the "pixel" mode, must be `auto` or a color. If using the "line" mode, must be `auto`, a stroke, or a [CeTZ style](#). If auto, defaults to the whole diagram's default style.

Default: `auto`